

Modularité en Python

Qu'est-ce que la Modularité ?

La modularité = **découper son programme en petits morceaux** (modules) indépendants.

Analogie

Imaginez une bibliothèque :

- ❑ Chaque **étagère** = un module
- ❑ Chaque **livre** = une fonction/variable
- ❑ Le **catalogue** = les imports

Pourquoi c'est utile ?

- ❑ Plus facile à **trouver** son code
- ❑ Plus simple à **corriger** les erreurs
- ❑ On peut **réutiliser** le même code dans plusieurs projets
- ❑ Travail à **plusieurs** sans se marcher sur les pieds

C'est Quoi un Module ?

Un module = un fichier Python

mon_premier_module.py ← C'est un module !

Exemple concret

```
# C'est mon premier module !

def dire_bonjour():
    print("Salut !")

def dire_bonsoir():
    print("Bonne soirée !")

auteur = "Moi-même"
```

Rien de compliqué ! Un simple fichier .py avec des fonctions dedans.

Comment Utiliser un Module ?

Pour utiliser un module, on l'importe !

Méthode 1 - Importer tout le module

```
import bienvenue

bienvenue.dire_bonjour() # Salut !
print(bienvenue.auteur)  # Moi-même
```

Méthode 2 - Importer seulement ce dont on a besoin

```
from bienvenue import dire_bonjour, auteur

dire_bonjour()      # Salut !
print(auteur)        # Moi-même
# dire_bonsoir()    # Erreur ! Non importée
```

Méthode 3 - Donner un surnom (alias)

```
import bienvenue as bv

bv.dire_bonjour()    # Salut !
```

À Quoi Sert `if __name__ == "__main__":` ?

Une variable magique : `__name__`

Dans `test.py` :

```
print(f"Je m'appelle {__name__}")
```

Quand on exécute directement :

```
python test.py  
# Affiche : Je m'appelle __main__
```

Quand on importe :

```
import test  
# Affiche : Je m'appelle test
```

L'astuce géniale

```
# mon_module.py  
def ma_fonction():  
    print("Je suis une fonction")  
  
# Ce code ne s'exécute que si on lance ce fichier directement  
if __name__ == "__main__":  
    print("Test de mon module")  
    ma_fonction()
```

Utilité : On peut tester son module sans que le test s'exécute quand on l'importe ailleurs !

Premier Projet Modulaire

Créons un mini projet : Calculatrice simple

```
calculatrice/  
├─ operations.py  
└─ main.py
```

```
# operations.py  
  
def addition(a, b):  
    return a + b  
  
def soustraction(a, b):  
    return a - b  
  
def multiplication(a, b):  
    return a * b  
  
def division(a, b):  
    if b == 0:  
        return "Erreur: division par zéro"  
    return a / b
```

```
# main.py  
import operations  
  
def afficher_menu():  
    print("\n--- CALCULATRICE ---")  
    print("1. Addition")  
    print("2. Soustraction")  
    print("3. Multiplication")  
    print("4. Division")  
    print("5. Quitter")  
  
def main():  
    while True:  
        afficher_menu()  
        choix = input("Choisissez une opération: ")  
  
        if choix == "5":  
            print("Au revoir !")  
            break  
  
        if choix in ["1", "2", "3", "4"]:  
            a = float(input("Premier nombre: "))  
            b = float(input("Deuxième nombre: "))  
  
            if choix == "1":  
                resultat = operations.addition(a, b)  
            elif choix == "2":  
                resultat = operations.soustraction(a, b)  
            elif choix == "3":  
                resultat = operations.multiplication(a, b)  
            else:  
                resultat = operations.division(a, b)  
  
            print(f"Résultat: {resultat}")  
        else:  
            print("Choix invalide")  
  
if __name__ == "__main__":  
    main()
```

Les Packages

Package = dossier qui contient des modules

À quoi ça sert ?

- regrouper plusieurs modules
- mieux organiser un projet
- créer une structure claire
- éviter de mettre tout dans un seul fichier
- faciliter la réutilisation du code

Structure :

```
mon_package/  
├── __init__.py    ← Fichier OBLIGATOIRE (peut être vide)  
├── module1.py  
├── module2.py  
└── sous_package/ ← Package dans un package  
    ├── __init__.py  
    └── module3.py
```

Le fichier `__init__.py` dit à Python : "Ce dossier est un package !"

```
import mon_package.module1  
from mon_package import module2  
from mon_package.sous_package import module3
```

Exemple de Package

```
utils/  
├── __init__.py  
├── texte.py  
└── maths.py
```

utils/texte.py :

```
def majuscules(texte):  
    return texte.upper()  
  
def minuscules(texte):  
    return texte.lower()  
  
def compter_mots(texte):  
    return len(texte.split())
```

utils/maths.py :

```
def carre(n):  
    return n ** 2  
  
def cube(n):  
    return n ** 3  
  
def est_pair(n):  
    return n % 2 == 0
```

Utiliser un Package

Méthode 1 : Importer un module du package

```
from utils import texte  
  
print(texte.majuscules("bonjour")) # BONJOUR  
print(texte.compter_mots("J'aime Python")) # 3
```

Méthode 2 : Importer directement une fonction

```
from utils.texte import majuscules, minuscules  
  
print(majuscules("python")) # PYTHON  
print(minuscules("PYTHON")) # python
```

Méthode 3 : Importer plusieurs modules

```
from utils import texte, maths  
  
print(texte.majuscules("hello")) # HELLO  
print(maths.carre(5)) # 25
```

Imports Relatifs

Dans un package, on peut importer entre modules

Dans module_a.py

```
mon_package/  
├── __init__.py  
├── module_a.py  
└── module_b.py
```

```
# module_a.py  
def fonction_a():  
    return "A"  
  
def fonction_commune():  
    return "Commune A"
```

Dans module_b.py :

```
# module_b.py  
from . import module_a      # . = même dossier  
from .module_a import fonction_a  
  
def fonction_b():  
    return "B"  
  
def fonction_commune():  
    return "Commune B"  
  
def utiliser_a():  
    return fonction_a() + " + " + fonction_b()
```

Les imports relatifs ne marchent que dans les packages !

Projet Complet - Gestionnaire de Tâches

```
todo/  
├─ main.py      # Programme principal  
├─ taches.py    # Gestion des tâches  
├─ fichiers.py  # Sauvegarde/chargement  
└─ affichage.py # Affichage des menus
```

```
# fichiers.py  
import json  
  
def sauvegarder_taches(taches, nom_fichier="taches.json"):  
    """Sauvegarde les tâches dans un fichier JSON"""  
    try:  
        with open(nom_fichier, 'w') as f:  
            json.dump(taches, f)  
        return True  
    except:  
        return False  
  
def charger_taches(nom_fichier="taches.json"):  
    """Charge les tâches depuis un fichier JSON"""  
    try:  
        with open(nom_fichier, 'r') as f:  
            return json.load(f)  
    except:  
        return [] # Retourne liste vide si erreur
```

```
# taches.py  
taches = [] # Liste globale des tâches  
  
def ajouter_tache(titre):  
    """Ajoute une nouvelle tâche"""  
    taches.append({"titre": titre, "faite": False})  
    return True  
  
def supprimer_tache(index):  
    """Supprime une tâche par son index"""  
    if 0 <= index < len(taches):  
        del taches[index]  
        return True  
    return False  
  
def marquer_faite(index):  
    """Marque une tâche comme terminée"""  
    if 0 <= index < len(taches):  
        taches[index]["faite"] = True  
        return True  
    return False  
  
def obtenir_taches():  
    """Retourne la liste des tâches"""  
    return taches
```


Projet Complet - Gestionnaire de Tâches

```
# affichage.py
def afficher_menu():
    """Affiche le menu principal"""
    print("\n" + "="*40)
    print("    GESTIONNAIRE DE TÂCHES")
    print("="*40)
    print("1. Ajouter une tâche")
    print("2. Voir les tâches")
    print("3. Marquer une tâche comme faite")
    print("4. Supprimer une tâche")
    print("5. Sauvegarder")
    print("6. Charger")
    print("7. Quitter")
    print("="*40)

def afficher_taches(taches):
    """Affiche toutes les tâches"""
    if not taches:
        print("\nAucune tâche pour le moment")
        return

    print("\nVOS TACHES :")
    for i, tache in enumerate(taches):
        statut = "[X]" if tache["faite"] else "[ ]"
        print(f"{i+1}. {statut} {tache['titre']}")
```

Projet Complet - Gestionnaire de Tâches

```
from taches import *
from fichiers import sauvegarder_taches, charger_taches
from affichage import afficher_menu, afficher_taches

def main():
    """Programme principal"""
    # On charge les tâches existantes
    global taches
    taches = charger_taches()

    while True:
        afficher_menu()
        choix = input("\nVotre choix : ")

        if choix == "1":
            titre = input("Titre de la tâche : ")
            ajouter_tache(titre)
            print(f"[OK] Tâche '{titre}' ajoutée !")

        elif choix == "2":
            afficher_taches(taches)

        elif choix == "3":
            afficher_taches(taches)
            try:
                index = int(input("Numéro de la tâche à terminer : "))
                if marquer_faite(index):
                    print("[OK] Tâche marquée comme faite !")
                else:
                    print("[ERREUR] Index invalide")
            except ValueError:
                print("[ERREUR] Veuillez entrer un nombre")

        elif choix == "4":
            afficher_taches(taches)
            try:
                index = int(input("Numéro de la tâche à supprimer : ")) - 1
                if supprimer_tache(index):
                    print("[SUPPRIME] Tâche supprimée !")
                else:
                    print("[ERREUR] Index invalide")
            except ValueError:
                print("[ERREUR] Veuillez entrer un nombre")

        elif choix == "5":
            if sauvegarder_taches(taches):
                print("[SAUVEGARDE] Sauvegarde réussie !")
            else:
                print("[ERREUR] Erreur lors de la sauvegarde")

        elif choix == "6":
            taches = charger_taches()
            print("[CHARGEMENT] Chargement réussi !")

        elif choix == "7":
            # Sauvegarde automatique avant de quitter
            sauvegarder_taches(taches)
            print("[AU REVOIR] Au revoir !")
            break

        else:
            print("[ERREUR] Choix invalide")

if __name__ == "__main__":
```

Modules Python Intégrés

```
import math

print(math.sqrt(16))      # 4.0 (racine carrée)
print(math.pi)            # 3.14159...
print(math.floor(3.7))    # 3 (arrondi inférieur)
print(math.ceil(3.2))     # 4 (arrondi supérieur)
```

```
import random

print(random.randint(1, 10)) # Nombre entre 1 et 10
print(random.choice(['a', 'b', 'c'])) # Choix aléatoire
```

```
import os

print(os.getcwd())        # Dossier actuel
print(os.listdir('.'))    # Liste des fichiers
os.mkdir('nouveau_dossier') # Créer un dossier
```

```
import datetime

print(datetime.datetime.now()) # Date et heure actuelles
print(datetime.date.today())  # Date du jour
```

Module **os** - Interface avec le système d'exploitation

```
import os

# Lister le contenu du dossier actuel
contenu = os.listdir(".")
print("Contenu du dossier :")
for element in contenu:
    print(f"  - {element}")

# Lister un dossier spécifique
fichiers = os.listdir("/home/utilisateur/Documents")
```

```
import os

# Créer un dossier
os.mkdir("nouveau_dossier")
print("Dossier créé")

# Créer une arborescence complète
os.makedirs("dossier1/sous_dossier2/sous_sous_dossier3")

# Supprimer un dossier VIDE
os.rmdir("nouveau_dossier")

# Supprimer une arborescence (dossiers vides)
os.removedirs("dossier1/sous_dossier2/sous_sous_dossier3")
```

```
import os

# Obtenir le répertoire de travail actuel
chemin_actuel = os.getcwd()
print(f"Je suis dans : {chemin_actuel}")

# Changer de répertoire
os.chdir("/chemin/vers/dossier")
print(f"Nouveau répertoire : {os.getcwd()}")

# Revenir au dossier parent
os.chdir("..")
```

```
import os

# Supprimer un fichier
if os.path.exists("fichier.txt"):
    os.remove("fichier.txt")
    print("Fichier supprimé")
```

Module os - Commandes système

```
import os

# Exécuter une commande système
os.system("ls -l") # Linux/Mac
os.system("dir")   # Windows

# Alternative plus moderne (recommandée)
import subprocess

# Capturer la sortie
resultat = subprocess.run(["ls", "-l"], capture_output=True, text=True)
print(resultat.stdout)

# Avec gestion d'erreur
try:
    subprocess.run(["commande_inexistante"], check=True)
except subprocess.CalledProcessError as e:
    print(f"Erreur : {e}")
```

```
import sys

# sys.argv contient les arguments passés au script
print(f"Nom du script : {sys.argv[0]}")
print(f"Arguments : {sys.argv[1:]}")

# Exemple d'utilisation
if len(sys.argv) > 1:
    print(f"Bonjour {sys.argv[1]}!")
else:
    print("Donnez un nom en argument")
```

```
python mon_script.py Alice
# Affiche : Bonjour Alice!
```

```
import sys

# Sortie standard
sys.stdout.write("Message normal\n")

# Sortie d'erreur
sys.stderr.write("Message d'erreur\n")

# Quitter le programme
sys.exit(0)      # Succès
sys.exit(1)      # Erreur
```

